

empty

UCS 503 - Software Engineering

ICU Patient Face Monitoring System

**A Computer-Vision-Based Non-Contact System for Continuous Facial
and Ocular Monitoring of Intensive Care Unit Patients**

Submitted by:

1024240103 Pahul Singh

1024240039 Harjot Singh

BE Third Year, Data Science and Artificial Intelligence

Submitted to:

Ms. Anushka, UCS 503

Project Proposal

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	4
1.1	Background and Context	4
1.2	Problem Statement	4
1.3	Project Objectives	5
2	Methodology and Design	6
2.1	System Architecture	6
2.1.1	High-Level Design	6
2.2	Technical Stack	7
2.3	Implementation Methodology	7
2.3.1	Face Detection	7
2.3.2	Facial Landmark Detection	7
2.3.3	Eye Landmark Extraction	8
2.3.4	Eye Feature Extraction	8
2.3.5	Eye-State Analysis	8
2.3.6	Blink Analysis	9
2.3.7	Eye Movement and Gaze Analysis	9
2.3.8	Temporal Processing	9
2.3.9	Global Value Generation	10
2.3.10	Alert Generation	10
3	System Requirements	11
3.1	Hardware Requirements	11
3.2	Software Requirements	11
3.3	Functional Requirements	11
3.4	Non-Functional Requirements	12
3.4.1	Real-Time Performance	12
3.4.2	Reliability	12
3.4.3	Accuracy	12
3.4.4	Maintainability	12
3.4.5	Privacy	12
4	Data Processing and Alert System	13
4.1	Data Flow	13
4.2	Raw Data	13
4.3	Extracted Data	14
4.4	Processed Data	14
4.5	Global Values	15
4.6	Meaningful Alerts	15

5	Expected Results	17
5.1	Expected Processing Output	17
6	Applications	18
7	Advantages and Limitations	19
7.1	Advantages	19
7.2	Limitations	19
8	Testing and Evaluation	20
8.1	Testing Methodology	20
8.1.1	Face Detection Testing	20
8.1.2	Eye Detection Testing	20
8.1.3	Feature Testing	20
8.1.4	Temporal Testing	21
8.1.5	Alert Testing	21
8.1.6	End-to-End Testing	21
8.2	Evaluation Metrics	22
9	Project Timeline	23
10	Future Scope	24
11	Conclusion	25

Introduction

1.1 Background and Context

Intensive Care Units (ICUs) contain patients who require continuous observation because their physical and neurological condition can change rapidly. Traditional patient monitoring systems primarily depend on sensors attached to the patient, such as ECG electrodes, pulse oximeters, blood-pressure cuffs, and other clinical devices. Although these systems provide important physiological measurements, they do not directly describe visible facial and ocular behaviour.

Computer vision provides a non-contact method for observing visible patient characteristics through a camera. A camera can capture facial information without requiring additional physical sensors to be attached to the patient’s face. Facial landmarks can then be used to locate important regions such as the eyes and determine geometric features associated with eye opening, closing, blinking, and eye movement.

Vision-based patient monitoring has been studied as a possible supplement to conventional monitoring systems [BDMS16; DMS+19]. The purpose of this project is to develop a focused Face + Eye monitoring pipeline that converts camera frames into structured information that can support continuous ICU observation.

The system is designed as a supplementary monitoring and decision-support system. It is not intended to replace medical professionals or provide an independent medical diagnosis.

1.2 Problem Statement

Continuous visual observation of an ICU patient can be difficult when healthcare staff must simultaneously supervise multiple patients. Important changes in facial and ocular behaviour may not always be immediately noticed.

The project therefore addresses the following problem:

How can facial and eye information obtained from a standard camera be automatically processed into meaningful, time-based monitoring values and alerts for an ICU patient without requiring physical contact with the patient’s face?

The proposed system focuses specifically on the face and eye regions. It detects the patient’s face, identifies facial and ocular landmarks, extracts geometric features, analyses their behaviour over time, and converts the resulting information into global monitoring values and meaningful alerts.

1.3 Project Objectives

The main objectives of the project are:

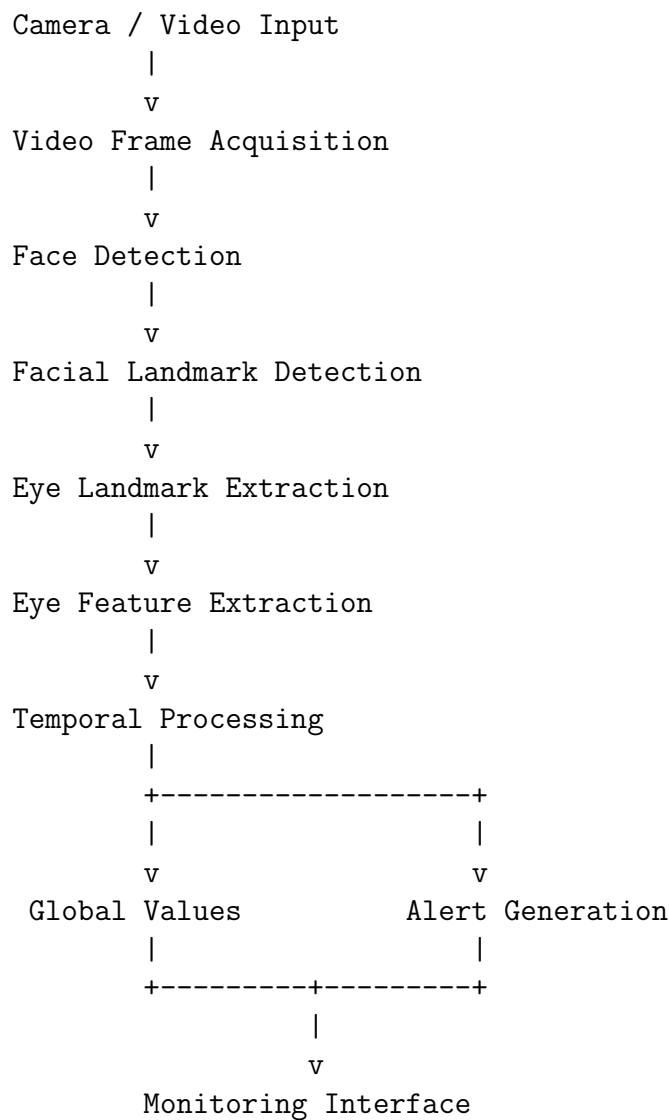
1. Detect a patient's face from a live or recorded camera stream.
2. Locate facial landmarks required for accurate eye-region analysis.
3. Extract landmarks corresponding to the left and right eyes.
4. Calculate geometric eye features such as the Eye Aspect Ratio (EAR).
5. Identify changes in eye state over time.
6. Analyse eye closure and blinking behaviour using temporal information.
7. Extract useful information related to eye movement and approximate gaze behaviour where reliable landmarks are available.
8. Convert frame-level measurements into stable temporal values.
9. Generate global monitoring values representing the current state of the facial and ocular observations.
10. Generate meaningful alerts when predefined abnormal or noteworthy patterns are detected.
11. Provide processed values to the monitoring interface so that the information can be displayed in an understandable form.

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

The proposed system follows a sequential computer-vision pipeline. The major stages are:



The camera provides a continuous sequence of frames. Each frame is processed to locate the patient's face and facial landmarks. The eye landmarks are then used

to calculate ocular features. These frame-level measurements are not directly treated as final outputs. Instead, they are processed over time to reduce noise and identify meaningful patterns.

The processed information is converted into global values and alert states. These results are then supplied to the monitoring interface, where they can be presented to a user in an understandable format.

2.2 Technical Stack

The proposed implementation uses the following technologies:

Table 2.1: Proposed Technical Stack

Technology	Purpose
Python	Main programming language for the computer-vision pipeline
OpenCV	Video capture, frame processing and image operations
MediaPipe Face Mesh	Facial landmark detection and face geometry
NumPy	Numerical calculations and feature processing
Flask/FastAPI	Optional service layer for delivering processed values
Browser Interface	Display of monitoring values and alerts
Git	Version control and project collaboration

OpenCV provides a widely used computer-vision framework for image and video processing. MediaPipe Face Mesh provides real-time facial landmark information suitable for extracting geometric information from the face [KAGG19; Ope26; Goo26].

2.3 Implementation Methodology

2.3.1 Face Detection

The first stage of the pipeline identifies the patient’s face in each incoming frame.

The face detection stage produces a region of interest containing the face. Processing can then be restricted primarily to this region, reducing unnecessary computation from the background.

Traditional face detection methods such as the Viola–Jones framework demonstrated the possibility of real-time face detection using visual features and classifiers [VJ04]. Modern landmark-based approaches can provide additional geometric information required by the remaining stages.

2.3.2 Facial Landmark Detection

After locating the face, facial landmarks are detected.

A facial landmark is a point corresponding to a meaningful position on the face. Multiple landmarks collectively describe the geometry of the face. The eye regions can be extracted from these landmarks.

The landmark stage provides a common coordinate representation that can be used for further measurements instead of relying only on raw pixel appearance.

2.3.3 Eye Landmark Extraction

The left and right eye regions are extracted from the complete set of facial landmarks.

For each eye, the relevant landmarks describe the upper eyelid, lower eyelid, and horizontal eye boundaries. These points provide the geometric information required for estimating whether the eye is open, partially closed, or closed.

2.3.4 Eye Feature Extraction

One of the primary geometric features is the Eye Aspect Ratio (EAR). EAR provides a numerical representation of the vertical opening of an eye relative to its horizontal width.

For six eye landmarks, EAR can be represented as:

$$EAR = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2 \|p_1 - p_4\|} \quad (2.1)$$

where $p_1, \dots, p_6$ represent selected eye landmarks and $\|\cdot\|$ represents Euclidean distance.

When an eye is open, the vertical distances are generally larger. When the eye closes, these distances decrease. This makes EAR useful for analysing eye closure and blinking behaviour [SČ16].

2.3.5 Eye-State Analysis

The calculated eye features can be converted into an eye-state classification.

A simplified representation is:

Condition	Possible Eye State
EAR above threshold	Open
EAR near threshold	Partially closed
EAR below threshold	Closed

The threshold should not be considered a universal medical value. It may depend on camera position, patient characteristics, lighting, facial orientation, and the specific implementation.

2.3.6 Blink Analysis

A blink can be interpreted as a short-duration transition from an open-eye state to a closed-eye state and then back to an open-eye state.

The system can maintain a temporal sequence of eye states:

Open -> Closing -> Closed -> Opening -> Open

This sequence can be used to identify individual blink events.

The duration and frequency of such events can be calculated from the sequence of observations. Real-time landmark-based blink detection has previously been investigated using facial landmark geometry [SC16].

2.3.7 Eye Movement and Gaze Analysis

Facial and eye landmarks can also provide information about relative eye movement.

Instead of attempting to determine an exact medical gaze direction, the system can represent approximate movement categories such as:

- left,
- right,
- upward,
- downward,
- approximately centred,
- uncertain.

The reliability of this information depends on camera placement, head orientation, illumination, eyelid visibility, and landmark quality.

2.3.8 Temporal Processing

Frame-by-frame measurements can be noisy. A single incorrect landmark position should therefore not immediately generate an alert.

The system maintains a short temporal history of observations.

For a sequence:

$$x_1, x_2, \dots, x_n$$

a simple moving average can be calculated as:

$$\bar{x}_t = \frac{1}{N} \sum_{i=0}^{N-1} x_{t-i} \quad (2.2)$$

where N is the number of recent observations.

Temporal processing can therefore make the output more stable and reduce false alerts caused by individual noisy frames.

2.3.9 Global Value Generation

The system converts multiple low-level measurements into a smaller set of global monitoring values.

Examples include:

- current eye state,
- left-eye EAR,
- right-eye EAR,
- average EAR,
- blink count,
- recent blink frequency,
- eye-closure duration,
- approximate gaze state,
- confidence or validity state,
- current alert state.

These values provide a compact representation of the patient's recent facial and ocular observations.

2.3.10 Alert Generation

Alerts are generated from processed temporal values rather than individual raw frames.

For example, if the eye remains below the closure threshold for a predefined period, the system can generate a prolonged-eye-closure alert.

A conceptual rule is:

$$Alert = \begin{cases} 1, & \text{if closure duration} > T_c \\ 0, & \text{otherwise} \end{cases} \quad (2.3)$$

where T_c represents a configurable duration threshold.

Similarly, abnormal or noteworthy changes in blink behaviour can be represented by additional rules.

These alerts are intended to act as supplementary monitoring indicators rather than clinical diagnoses.

System Requirements

3.1 Hardware Requirements

The proposed system requires:

- A standard RGB camera or webcam.
- A computer or workstation capable of real-time video processing.
- Sufficient RAM for the operating system and computer-vision pipeline.
- Optional GPU acceleration for higher-resolution or multi-stream processing.
- Stable camera positioning near the patient's bed.

3.2 Software Requirements

Table 3.1: Software Requirements

Component	Requirement
Operating System	Windows/Linux/macOS or equivalent environment
Programming Language	Python
Computer Vision	OpenCV
Landmark Detection	MediaPipe Face Mesh
Numerical Processing	NumPy
Backend	Flask or FastAPI, if required
Interface	Browser-based monitoring interface
Version Control	Git

3.3 Functional Requirements

The system shall:

1. Accept live or recorded video input.
2. Detect the patient's face.
3. Detect facial landmarks.

4. Identify eye landmarks.
5. Calculate eye-related geometric features.
6. Estimate eye state.
7. Detect blink events.
8. Maintain temporal information.
9. Calculate global monitoring values.
10. Generate configurable alerts.
11. Make processed values available to the monitoring interface.

3.4 Non-Functional Requirements

3.4.1 Real-Time Performance

The pipeline should process frames with sufficiently low latency for continuous monitoring. The target is to maintain approximately 15 frames per second or higher where the available hardware permits.

3.4.2 Reliability

The system should tolerate temporary landmark loss, lighting variation, and small head movements without crashing or generating continuous false alerts.

3.4.3 Accuracy

The system should be evaluated using labelled video sequences and appropriate metrics rather than assuming that every detected condition is correct.

3.4.4 Maintainability

The face detection, eye analysis, temporal processing, alert generation, and interface communication components should remain modular so that individual components can be improved independently.

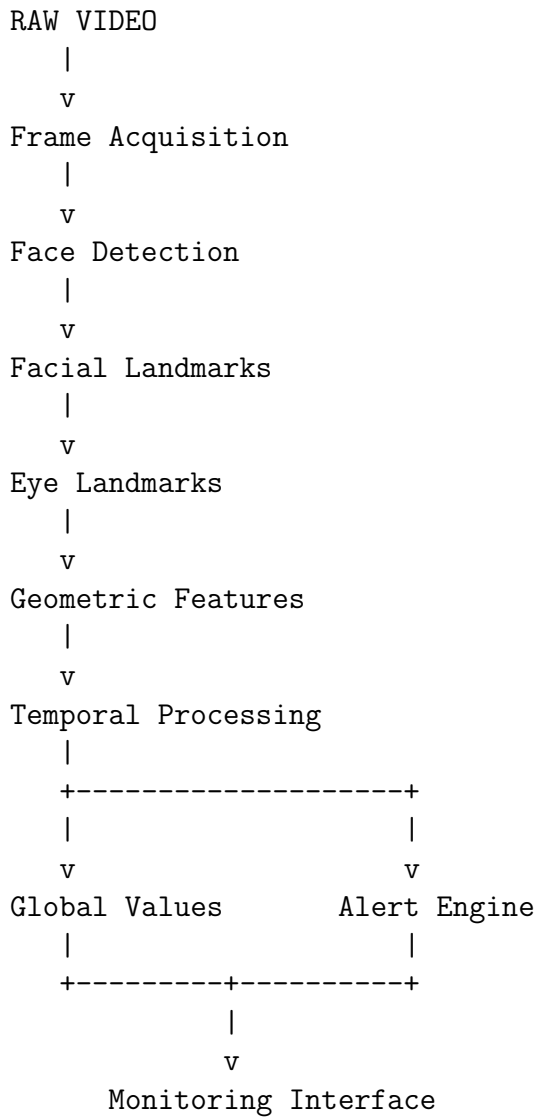
3.4.5 Privacy

Patient video represents sensitive information. The system should prefer local processing where possible and should avoid unnecessary storage or transmission of raw video.

Data Processing and Alert System

4.1 Data Flow

The complete data flow can be represented as:



4.2 Raw Data

Raw input consists primarily of camera frames.

Each frame contains pixel information representing the patient's visible face and surrounding environment.

Raw frames may also contain irrelevant information such as:

- background objects,
- bedding,
- medical equipment,
- other visual noise.

The processing pipeline therefore extracts only the information needed for face and eye monitoring.

4.3 Extracted Data

The face and landmark stages produce structured information such as:

- face location,
- facial landmark coordinates,
- eye landmark coordinates,
- eye-region geometry,
- detection confidence or validity information.

This information is significantly smaller and more structured than the original video frame.

4.4 Processed Data

The extracted measurements are processed to generate:

- left-eye EAR,
- right-eye EAR,
- average eye opening,
- eye state,
- blink events,
- closure duration,

- temporal statistics,
- approximate eye movement,
- confidence information.

Temporal processing is important because an individual frame may contain an incorrect or incomplete observation.

4.5 Global Values

Global values summarize the current condition of the Face + Eye pipeline.

A possible global monitoring record can be represented as:

```
{
  eye_state,
  left_ear,
  right_ear,
  average_ear,
  blink_count,
  closure_duration,
  gaze_state,
  confidence,
  alert_state
}
```

The exact set of fields can be changed according to the final implementation.

4.6 Meaningful Alerts

The purpose of the alert system is to convert numerical measurements into understandable monitoring information.

Possible alert categories include:

Table 4.1: Example Alert Categories

Alert	Meaning
Prolonged Eye Closure	Eye closure persists beyond a configurable duration
Repeated Closure	Frequent closure events are observed in a recent time window
Unusual Blink Pattern	Blink behaviour differs from a configured baseline or rule
Low Confidence	Face or eye landmarks are unreliable
Face Not Detected	The patient's face cannot currently be located

Alerts should contain both the event and sufficient context for a user to understand why the alert was produced.

Expected Results

5.1 Expected Processing Output

The expected output of the Face + Eye pipeline is a continuously updated collection of structured monitoring values.

For every processed observation, the system should be able to provide information similar to:

Table 5.1: Expected Monitoring Output

Output	Description
Face Status	Whether a usable face is currently detected
Eye State	Open, partially closed, closed, or uncertain
EAR	Numerical measure of eye opening
Blink Information	Detection/count of blink events
Closure Duration	Duration of continuous or recent closure
Gaze State	Approximate relative eye movement state
Confidence	Reliability of the current observation
Alert	Current meaningful monitoring alert

The final processed values are intended to be sent to the monitoring interface. The interface can then convert these values into readable indicators, status cards, charts, or alerts.

Applications

The proposed Face + Eye monitoring system may be useful in:

1. **ICU Monitoring:** Supplementary observation of facial and ocular behaviour.
2. **Continuous Observation:** Automated monitoring can provide a persistent stream of computer-generated observations.
3. **Remote Monitoring:** Processed values may be made available to an authorised monitoring interface.
4. **Research:** The system can provide structured facial and eye measurements for research involving patient behaviour.
5. **Alert Support:** The system can highlight potentially noteworthy changes for further human attention.

The system should always be treated as a supplementary tool and not as a replacement for professional medical judgement.

Advantages and Limitations

7.1 Advantages

- **Non-contact:** No physical sensor is required on the patient's face.
- **Continuous:** Video can be processed continuously rather than through occasional manual observations.
- **Automated:** Facial and ocular measurements can be calculated automatically.
- **Structured Output:** Raw visual information is converted into numerical values and understandable alerts.
- **Modular:** Individual processing stages can be independently improved.
- **Interface Integration:** Processed global values can be supplied to a monitoring interface.

7.2 Limitations

The system has several limitations:

- Poor illumination can reduce landmark accuracy.
- Significant head rotation can make eye landmarks unreliable.
- Occlusion caused by blankets, equipment, hands, or other objects can interfere with detection.
- Camera placement affects the quality of measurements.
- Individual differences in facial structure can affect threshold-based eye-state classification.
- A computer-vision alert does not necessarily indicate a medical emergency.
- Raw video may contain sensitive patient information and therefore requires appropriate privacy controls.

Testing and Evaluation

8.1 Testing Methodology

The system should be evaluated component by component before performing complete end-to-end testing.

8.1.1 Face Detection Testing

Face detection should be tested under:

- different lighting conditions,
- different face orientations,
- different distances from the camera,
- partial occlusion,
- periods of temporary detection failure.

8.1.2 Eye Detection Testing

Eye landmarks should be evaluated for:

- open eyes,
- closed eyes,
- blinking,
- partially visible eyes,
- head movement,
- varying illumination.

8.1.3 Feature Testing

EAR calculations should be checked against manually labelled eye states.

The calculated values should behave consistently with the expected geometric changes of the eye.

8.1.4 Temporal Testing

Temporal processing should be tested using sequences containing:

- normal eye opening,
- individual blinks,
- multiple blinks,
- prolonged closure,
- temporary landmark loss.

8.1.5 Alert Testing

Alert generation should be evaluated using controlled test sequences.

The test should determine whether:

1. valid events generate alerts,
2. normal observations avoid unnecessary alerts,
3. temporary detection errors do not create excessive alerts,
4. alert states return to normal when the condition ends.

8.1.6 End-to-End Testing

Finally, the complete system should be tested from camera input to monitoring interface.

The complete path is:

```
Camera
-> Frame
-> Face
-> Landmarks
-> Eye Features
-> Temporal Processing
-> Global Values
-> Alerts
-> Monitoring Interface
```

8.2 Evaluation Metrics

The following metrics can be used during evaluation:

Table 8.1: Proposed Evaluation Metrics

Metric	Purpose
Detection Accuracy	Measures correct face/eye detections
Precision	Measures proportion of predicted events that are correct
Recall	Measures proportion of actual events that are detected
F1 Score	Balances precision and recall
Latency	Measures processing delay
FPS	Measures real-time processing throughput
False Alert Rate	Measures unnecessary alerts
Miss Rate	Measures important events that were not detected

The project may use targets such as:

- latency below approximately 50 ms per processing step where hardware permits,
- face detection accuracy above 95% on the selected test data,
- blink detection accuracy above 90% on labelled sequences,
- processing throughput of at least approximately 15 FPS.

These values are proposed evaluation targets and should not be presented as achieved performance until experimental testing has confirmed them.

Project Timeline

Table 9.1: Proposed Project Timeline

Phase	Work
Phase 1	Requirement analysis and literature review
Phase 2	Camera input and face detection
Phase 3	Facial and eye landmark extraction
Phase 4	Eye feature calculation and state analysis
Phase 5	Temporal processing and blink analysis
Phase 6	Global value and alert generation
Phase 7	Monitoring interface integration
Phase 8	Testing, evaluation and documentation

The phases can overlap where appropriate. For example, interface development can proceed while the computer-vision pipeline is being tested.

Future Scope

Future development can extend the system in several directions.

1. **Improved Personalisation:** Patient-specific baselines could be used instead of relying only on fixed thresholds.
2. **Advanced Temporal Models:** Machine-learning models could be used to identify more complex temporal patterns.
3. **Multi-Camera Monitoring:** Multiple camera views could improve robustness against occlusion and head rotation.
4. **Improved Gaze Estimation:** More sophisticated methods could provide more reliable relative gaze information.
5. **Better Uncertainty Handling:** Confidence-aware processing could prevent unreliable frames from influencing alerts.
6. **Historical Analysis:** Processed global values could be stored securely to allow authorised users to examine trends over time.
7. **Scalable Deployment:** The system could be extended to support multiple patient monitoring streams.

Conclusion

This project proposes a computer-vision-based Face + Eye monitoring system for continuous, non-contact observation of ICU patients.

The proposed pipeline begins with camera input and proceeds through face detection, facial landmark detection, eye landmark extraction, eye feature calculation, and temporal processing. The resulting information is converted into global monitoring values and meaningful alerts.

The final processed information can then be provided to a monitoring interface so that complex computer-vision measurements are presented in a simple and understandable form.

The system is intended to supplement existing patient monitoring rather than replace healthcare professionals. Its usefulness will ultimately depend on the quality of the camera input, robustness of landmark detection, accuracy of temporal analysis, appropriate threshold selection, and systematic experimental evaluation.

Bibliography

- [BDMS16] Vitoantonio Bevilacqua, Donato D’Ambruoso, Giacomo Mandolino, and Mariapia Suma. “Vision-Based Patient Monitoring: A Comprehensive Review of Algorithms and Technologies”. In: *Journal of Ambient Intelligence and Humanized Computing* (2016) (cit. on p. 4).
- [DMS+19] Anis Davoudi, Kumar Rohit Malhotra, Benjamin Shickel, et al. “Intelligent ICU for Autonomous Patient Monitoring Using Pervasive Sensing and Deep Learning”. In: *Scientific Reports* 9.1 (2019), p. 8020 (cit. on p. 4).
- [Goo26] Google MediaPipe. *MediaPipe Face Mesh*. <https://github.com/google-ai-edge/mediapipe>. 2026 (cit. on p. 7).
- [KAGG19] Yury Kartynnik, Artsiom Ablavatski, Ivan Grishchenko, and Matthias Grundmann. “Real-Time Facial Surface Geometry from Monocular Video on Mobile GPUs”. In: *arXiv preprint arXiv:1907.06724* (2019) (cit. on p. 7).
- [Ope26] OpenCV. *Open Source Computer Vision Library*. <https://opencv.org>. 2026 (cit. on p. 7).
- [SČ16] Tereza Soukupová and Jan Čech. “Real-Time Eye Blink Detection Using Facial Landmarks”. In: *21st Computer Vision Winter Workshop*. Rimske Toplice, Slovenia, 2016 (cit. on pp. 8, 9).
- [VJ04] Paul Viola and Michael J. Jones. “Robust Real-Time Face Detection”. In: *International Journal of Computer Vision* 57.2 (2004), pp. 137–154 (cit. on p. 7).